

المحاضرة الثالثة: طرق التعامل مع classes وطرق الوراثة في اطار Dart



م. سليمان الشوصي

محتويات المحاضرة Contents

❖ توضيح طريقة تكوين الكلاسات وانشاء الكائنات (Casses and Object)

❖ مفهوم تسلسل العمليات (Cascade Notation)

❖ الأعضاء الساكنة والمرجع الداخلي (static & this)

❖ التغليف وحماية البيانات (Getters & Setters)

❖ الوراثة والتسلسل الهرمي (Inheritance & extends)

❖ إعادة التعريف والمرجع الوراثةي (@override & super)

❖ تسلسل استدعاء دوال البناء (Constructor Chaining)

البرمجة كائنية التوجّه (Object-Oriented Programming – OOP)،

تُعد البرمجة كائنية التوجّه (OOP) أسلوباً أو نمطاً برمجياً (Programming Paradigm) يهدف إلى تنظيم الكود وحل المشكلات البرمجية عن طريق محاكاة عناصر العالم الحقيقي. بدلاً من كتابة البرامج على شكل تعليمات أو دوال متتالية ومنفصلة (كما في البرمجة الإجرائية التقليدية (Procedural Programming)، تقوم الـ OOP بتجميع البيانات والعمليات المتعلقة بها معاً داخل وحدات مستقلة ومتراصة يُطلق عليها الكائنات (Objects).

❖ تعتمد البرمجة كائنية التوجّه على مفهومي جوهرين:

الصف (Class): هو القالب النظري أو المخطط العام (Blueprint) الذي يُحدد الخصائص (Data/Attributes) والسلوكيات (Functions/Methods) التي ستمتّع بها الكائنات. **الكائن (Object):** هو التطبيق الفعلي (Instance) للـ Class؛ حيث يُحجز له مكان في ذاكرة الجهاز ليحمل بيانات حقيقية ومستقلة.

❖ المبادئ والأركان الأربعة الأساسية لـ OOP

١. **التغليف (Encapsulation):** تجميع البيانات والدوال داخل كلاس واحد، وحماية البيانات الداخلية من التعديل الخارجي المباشر إلا عبر طرق محددة وآمنة.
٢. **التجريد (Abstraction):** إخفاء تفاصيل التعقيد الداخلي وإظهار الواجهة والميزات الضرورية فقط للمستخدم النهائي للكلاس.
٣. **الوراثة (Inheritance):** إمكانية إنشاء كلاس جديد بناءً على كلاس سابق، لإعادة استخدام الخصائص والدوال وتجنب تكرار الكود.
٤. **تعدد الأشكال (Polymorphism):** قدرة الكائنات المختلفة على الاستجابة لنفس الدالة بأساليب وطرق مختلفة تناسب طبيعة كل كائن.

❖ مميزات البرمجة كائنية التوجّه:

- **إعادة استخدام الكود (Reusability):** بفضل الوراثة والـ Classes، لا نحتاج لتكرار بناء المكونات من الصفر.
- **تسهيل الصيانة والتطوير (Maintainability):** لأن البرنامج يكون مقسماً إلى وحدات مستقلة (Objects)؛ فتعديل جزء لا يخرّب بقية أجزاء النظام.
- **إدارة المشاريع الضخمة:** تعتبر الـ OOP العمود الفقري لأغلب أطر العمل الحديثة (مثل Flutter)، حيث توفر بيئة منظمة ومفهومة لفرق العمل المكونة من عدة مطورين.

```
class person {
  var name, age, phone;
}

Run | Debug
void main() {
  var x = new person();
  x.name = "suliman";
  x.phone = 77777777;
  x.age = 20;
  print(x.name);
  print(x.age);
  print(x.phone);
}
```

```
class person {
  var name, age, phone;
  ShowInfo() {
    print(name);
    print(age);
    print(phone);
  }
}

Run | Debug
void main() {
  var x = person();
  x.name = "suliman";
  x.phone = 77777777;
  x.age = 20;
  x.ShowInfo();
}
```

```
void main() {
  var x = person()
  ..name = "suliman"
  ..phone = 77777777
  ..age = 20
  ..ShowInfo();
}
```

❖ Class:

- في الكود: الكلاس person يمثل القالب الذي يحدد أن أي شخص في النظام سيمتلك (name, age, phone) ودالة ShowInfo().

❖ الكائن (Object):

- في الكود: المتغير x هو الـ Object الفعلي المشتق من الكلاس person وتم إسناد بيانات خاصة به (suliman, 20, 77777777).

وجه المقارنة	الصنف (Class)	الكائن (Object)
التبيين	مفهوم مجرد وقالب نظري (Blueprint).	كيان حقيقي وملموس (Instance).
الذاكرة	لا يشغل مساحة في الذاكرة للبيانات.	يشغل مساحة فعلية في الذاكرة (RAM).
الوجود في الكود	عام: يُكتب مرة واحدة فقط.	خاص: يمكن إنشاء عدة كائنات منه.

مفهوم تسلسل العمليات (Cascade Notation)

- هي ميزة برمجية مدمجة في لغة Dart نرمز لها بالرمز .. (Two Dots)، وتُعرف بـ Cascade Notation.
- آلية العمل: تتيح للمطور إمكانية إجراء سلسلة من العمليات المتتالية (تعديل متغيرات أو استدعاء دوال) على نفس الكائن (Object) دون الحاجة لإعادة كتابة اسم الكائن في كل سطر.

```
// مفهوم constructor
class person {
  var name, age, phone;
  person(p_name, p_age, p_phone) {
    name = p_name;
    age = p_age;
    phone = p_phone;
    print("name=$name");
    print("age=$age");
    print("phone=$phone");
  }
}

Run | Debug
void main() {
  var p1=person("suliman",20,7777);
}
```

```
// طريقة تمرير البيانات للكلاس
// constructor دالة من خلال
class person {
  person({p_name, p_age, p_phone}) {
    print("name=$p_name");
    print("age=$p_age");
    print("phone=$p_phone");
    print("*****");
  }
}

Run | Debug
void main() {
  var p1 = person();
  var p2 = person(p_name:"suliman",
                  p_age:20,p_phone: 7777);
}
```

```
name=null
age=null
phone=null
*****
name=suliman
age=20
phone=7777
*****
```

مفهوم دالة البناء (Constructor):

هو دالة خاصة (Special Method) داخل الصنف (Class)، تتميز بـ:

١. يحمل نفس اسم الكلاس تماماً (مثل person في الكود).
٢. لا يمتلك نوع إرجاع (لا نكتب قبله void أو أي نوع آخر).
٣. يُنفَّذ تلقائياً وفوراً عند إنشاء كائن جديد (Object) من الكلاس باستخدام الأقواس person().

وظيفته الأساسية:

- تُستخدم دالة البناء لتهيئة الكائن (Initialization) وتمرير البيانات الأولية للخصائص لحظة ولادة الكائن في الذاكرة.

الـ Constructor: هو الباب الرئيسي لإنشاء الكائن وتهيئته بالبيانات فور وإنشاءه.

الأقواس المجددة {} في الـ Constructor: تمنح المرونة لتمرير البيانات بأسمائها دون التقيد بالترتيب، أو تركها فارغة إن لم تكن إجبار.

```
class person {
  var name;
  var age;
  // لإشارة إلى متغيرات this دالة عادية تستخدم
  // الكائن الحالي
  ShowInfo() {
    print("name=${this.name}");
    print("age=${this.age}");
    print("*****");
  }
}

Run | Debug
void main() {
  var p1 = person();
  //إسناد قيم أولية للكائن p1
  p1.name = "khaled";
  p1.age = 24;
  p1.ShowInfo();
}
```

```
class person {
  var name;
  var age;
  ShowInfo({p_name, p_age}) {
    this.name = p_name;
    this.age = p_age;
    print("name=${this.name}");
    print("age=${this.age}");
    print("*****");
  }
}

Run | Debug
void main() {
  var p1 = person();
  p1.ShowInfo(p_name:"suliman",p_age:25);
}
```

مفهوم الكلمة المفتاحية this

هي مرجع (Reference) يشار به إلى النسخة الحالية من الكائن (Current Instance) المشتق من الكلاس أثناء وقت التشغيل (Runtime).

الفائدة والهدف الأساسي:

- فك الالتباس (Disambiguation): التمييز بين متغيرات الكلاس (Fields) وبين المتغيرات المحلية أو الوسائط (Parameters) إذا تلمست أسماءها.
- ربط البيانات: إسناد القيم الممررة في المعاملات مباشرة لخصائص الكائن الحالي.
- قاعدة ذهبية: **this** تُستخدم فقط مع الكائنات العادية، ولا يجوز كتابتها أو الاعتماد عليها داخل أي دالة أو متغير معرف بـ **static**.

```
//static مفهوم
class person {
  var name;
  static ShowInfo({p_name, p_age}) {
    print("name=$p_name");
    print("age=$p_age");
  }
}

Run | Debug
void main() {
  //استدعاء دالة الطباعة من خلال اسم الكلاس نفسه
  person.ShowInfo(p_name: "suliman");
}
```

```
class person {
  static var name;
  static ShowInfo({p_name}) {
    this.name = p_name;
    name = p_name;
    print("name=$name");
    print("*****");
  }
}

Run | Debug
void main() {
  person.ShowInfo(p_name: "suliman");
}
```

```
//طريقة الوصول الى المتغير static
//من خلال constructor
class person {
  static String? name;
  person({p_name}) {
    name = p_name;
    print("name=$name");
  }
}

Run | Debug
void main() {
  var p1 = new person(p_name: "Mohammed");
}
```

مفهوم الأعضاء الساكنة (static) في Dart

الكلمة المفتاحية **static** تُستخدم لتحويل الدالة أو المتغير من ملكية الكائن (Object Level) إلى ملكية الصنف نفسه (Class Level).

➤ **الدالة العادية (Non-Static):** ترتبط بكل كائن يُنشأ من الكلاس بشكل مستقل، وتتطلب إنشاء عينة (Object) لاستدعائها.

➤ **الدالة الساكنة (static):** ترتبط بالحاوية أو القالب الرئيسي (Class) مباشرة، وتُحفظ مرة واحدة فقط في الذاكرة بصرف النظر عن عدد الكائنات المنشأة.

الفائدة والهدف الأساسي:

- **الوصول المباشر:** استدعاء الدالة أو المتغير مباشرة بذكر اسم الكلاس دون الحاجة لإنشاء كائن جديد باستخدام `new` أو `person()`.
- **كفاءة الذاكرة (Memory Efficiency):** توفير مساحة الذاكرة؛ لأن الدالة الـ `static` تُنشأ مرة واحدة وتُشارك بين جميع أجزاء البرنامج بدلاً من تكرار حجز مساحة لها مع كل كائن جديد.
- **الدوال المساعدة (Utility Functions):** تُعد مثالية للدوال التي تقوم بمهمة عامة ولا تعتمد على حالة أو بيانات كائن معين (مثل الدوال الرياضية `Math.sqrt()` أو دوال الطباعة والتنسيق).

مفهوم الـ Getters and Setters

في البرمجة كائنية التوجّه، يُفضل دائماً حماية المتغيرات والبيانات الخاصة بالصنف (Class) ومنع الوصول إليها أو تعديلها بشكل مباشر من خارج الكلاس.

ولتحقيق ذلك، يتم إنشاء "بوابات آمنة" للتحكم في قراءة البيانات وكتابتها:

- **المعيّن (Setter):** هو دالة أو طريقة كتابة تُستخدم لإسناد وتمرير قيمة جديدة إلى المتغير الخاص داخل الكائن.
- **المسترجع (Getter):** هو دالة أو طريقة قراءة تُستخدم لجلب واسترجاع قيمة المتغير الخاص لقراءته أو عرضه من خارج الكائن.

لغرض والفائدة الأساسية من هذه الطريقة:

```
// طريقة ارسال وعرض البيانات باستخدام  
// Getters and Setters  
class person {  
  String? name;  
  void set(p_name) {  
    this.name = p_name;  
  }  
  get() {  
    print(name);  
  }  
}  
Run | Debug  
void main() {  
  var p1 = new person();  
  p1.set("Mohammed");  
  p1.get();  
}
```

```
//Dart مبدى الوراثة في ال
// Inheritance Override
class A {
  showInfo() {
    print("Indirect Super Class");
  }
}
class B extends A {
  showInfo2() {
    print("Direct Super Class");
  }
}
class C extends B {
  showInfo3() {
    print("Sub class");
  }
}
Run | Debug
void main() {
  var c = new C();
  c.showInfo();
  c.showInfo2();
  c.showInfo3();
}
```

❖ **الوراثة (Inheritance):** هي ركن أساسي في ال OOP يتيح لكلاس جديد (Subclass) أن يرث الخصائص والدوال من كلاس سابق قائم بالفعل (Superclass). هدفها الرئيسي: إعادة استخدام الكود (Code Reusability) وتجنب التكرار، وبناء هيكليّة منظمة ومتدرجة بين الكلاسات.

❖ **الكلمة المفتاحية extends** هي الأداة الرسمية في لغة Dart لإعلان علاقة الوراثة المباشرة بين كلاسين (علاقة is-a).

وظيفتها: تُخبر المحرر بأن الكلاس الابن يمتلك ويستطيع الوصول إلى جميع خصائص ودوال الكلاس الأب، إضافة إلى خصائصه الخاصة.

❖ **مسميات الكلاسات الوراثة والموروثة**

- **Sub Class (الكلاس الابن - Class C):** هو الكلاس الممتد والوارث، ويقع في أسفل الشجرة الهرمية. يمتلك أعلى قدر من الصلاحيات والوصول؛ لأنه يرث كلاً من أبيه المباشر وجده، ولذا استطاع استدعاء جميع الدوال الثلاث (showInfo, showInfo2, showInfo3) من كائن واحد c.
- **Direct Super Class (الأب المباشر - Class B):** هو الكلاس الذي يمتد منه الابن مباشرة عبر ال extends. أيه المباشر A ويورث خصائصه للابن C.
- **Indirect Super Class (الأب غير المباشر / الجد - Class A):** هو الكلاس الأعلى في قمة السلسلة الوراثة لهذا الكود. لا يرث من الكلاسات المذكورة تحته، ولكن خصائصه تصل إلى الابن C بفضل وسطية الأب المباشر B (التوريث المتسلسل).

```
Class A → Indirect Super Class (الأب غير المباشر / الجد)
  ↑
  | (extends)
Class B → Direct Super Class (الأب المباشر)
  ↑
  | (extends)
Class C → Sub Class (الابن / الكلاس المشتق)
```

```
//Override مبدى
class A {
  showInfo() {
    print("Suliman AlShossi");
  }
}
class B extends A {
  @override
  showInfo() {
    print("Mohammed Ali");
  }
}
class C extends B {}

Run | Debug
void main() {
  var a = A();
  a.showInfo();
  var b = B();
  b.showInfo();
  var c = C();
  c.showInfo();
}
```

```
Suliman AlShossi
Mohammed Ali
Mohammed Ali
```

❖ **مفهوم الـ Override (التجاوز / إعادة التعريف):** هو مبدأ يُتيح للكلاس الابن (Subclass) إعادة كتابة وتعديل سلوك دالة موروثه من الكلاس الأب (Superclass) لكي تؤدي وظيفة جديدة تتناسب مع احتياجات الابن، مع الحفاظ على نفس اسم الدالة ونفس مدخلاتها.

الفائدة منه:

- **تخصيص السلوك (Customization):** إعطاء الحرية للكلاسات الأبناء لتعديل طريقة عمل الدوال الموروثة دون تغيير الكلاس الأب.
- **تطبيق تعدد الأشكال (Polymorphism):** استدعاء نفس اسم الدالة على كائنات مختلفة، حيث ينفذ كل كائن النسخة الخاصة به.

❖ **الوسم @override:** هو علامة إرشادية للمترجم (Compiler) تُكتب فوق الدالة المعدلة للتأكد من أن الدالة موجودة بالفعل في الكلاس الأب، ولتنبيه المطور عند وجود أي خطأ في كتابة اسم الدالة.

```
//super and this مبدى
class A {
  var phone;
  showInfo1(phone) {
    print(phone);
    print(this.phone);
  }
}
class B extends A {
  var phone;
  showInfo() {
    this.phone = 777777777;
    super.phone = 7888888888;
    super.showInfo1(this.phone);
  }
}
class C extends B {}
Run | Debug
void main() {
  var c = C();
  c.showInfo();
}
```

```
777777777
7888888888
```

❖ **الكلمة المفتاحية this (المرجع الداخلي):** هي لمفهوم: هي مرجع (Reference) يشير إلى النسخة الحالية من الكائن (Current Instance) داخل الكلاس نفسه.

الفائدة منه:

- الوصول لمتغيرات ودوال نفس الكلاس الحالي.
- التمييز وفك التداخل بين متغيرات الكلاس (Fields) وبين المعاملات المحلية (Parameters) عندما تتشابه أسمائها.

❖ **الكلمة المفتاحية super (المرجع الوراثي):** هي مرجع تشير به الدالة إلى الكلاس الأب المباشر (Direct Superclass) في السلسلة الوراثية.

❖ الفائدة منه:

- الوصول إلى أعضاء (متغيرات ودوال) الكلاس الأب من داخل الكلاس الابن.
- استدعاء دالة البناء (Constructor) الخاصة بالأب عند بناء الكائن الابن.
- تشغيل النسخة الأصلية لدالة في الأب تم إعادة تعريفها (@override) في الابن.

```
//كيفية التعامل مع ال Constructor
// في الوراثة
class A {
  A() {
    print("name_A = Mohammed");
  }
}
class B extends A {
  B() {
    print("name_B= suliman");
  }
}
Run | Debug
void main() {
  var b = B();
}
```

```
name_A = Mohammed
name_B= suliman
```

❖ **استدعاء وتمرير البيانات بين دوال البناء (Constructor Chaining / Super Constructor Call):** ي آلية برمجية في البرمجة كائنية التوجه (OOP) تتيح لدالة بناء الكلاس الابن (Subclass Constructor) استدعاء وتمرير البيانات تلقائياً أو صراحةً إلى دالة بناء الكلاس الأب (Superclass Constructor) المباشر أثناء عملية إنشاء الكائن في الذاكرة.

❖ الغرض والفائدة الأساسية:

- **التهيئة المتسلسلة (Initialization Chain):** ضمان بناء وتأمين أجزاء الكلاس الأب وتهيئة خصائصه أولاً قبل إكمال بناء خصائص الابن.
- **تمرير البيانات (Data Passing):** إمكانية إرسال المدخلات والوسائط المطلوبة لخصائص الأب الموروثة لحظة ولادة الكائن الابن عبر الصيغة البرمجية : `super(...)`.
- **تجنب تكرار الكود (DRY Principle):** عدم إعادة كتابة منطق تهيئة البيانات الأساسية في الكلاس الابن، والاعتماد على دالة البناء القائمة بالفعل في الكلاس الأب.

ملاحظة (قاعدة) : عند إنشاء كائن من الابن، تسير البيانات صعوداً للأب لتنفيذ دالة بنائه أولاً، ثم ينتهي التنفيذ نزولاً في دالة بناء الابن.

❖ استدعاء وتمرير البيانات بين دوال البناء (Constructor Chaining / Super Constructor Call):

الاستدعاء والتمرير (من الأسفل إلى الأعلى):

١. الكلاس الابن C:

• يستقبل القيمة "suliman" في المتغير المحلي name.

• يقوم الـ Constructor في C بتمرير قيمة ثابتة هي "Mohammed" للأب المباشر عبر : super("Mohammed").

٢. الكلاس الأب المباشر B:

• يستقبل القيمة الممررة من الابن وهي "Mohammed" في المتغير name.

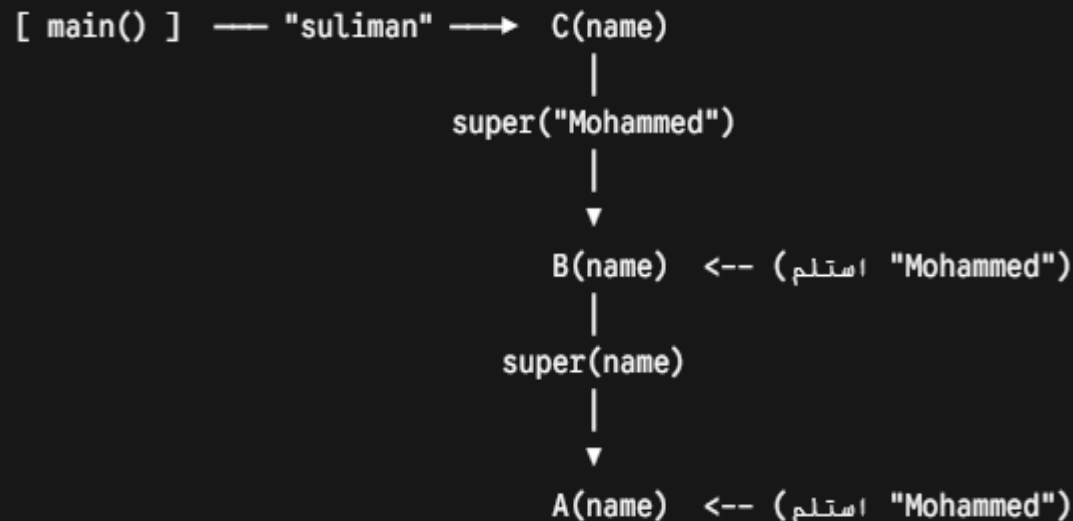
• يقوم بدوره بتمرير نفس هذه القيمة للأب المالي عبر : super(name).

٣. الكلاس الأب الأعلى A:

• يستقبل القيمة "Mohammed" في المتغير name.

```
// كيفية التعامل مع الـ
// في الوراثة
class A {
  A(name) {
    print("name_A = $name");
  }
}
class B extends A {
  B(name) : super(name) {
    print("nameB = $name");
  }
}
class C extends B {
  C(name) : super("Mohammed") {
    print("name_C = $name");
  }
}
Run | Debug
void main() {
  var c = C("sulinman");
}
```

```
name_A =Mohammed
name_B =Mohammed
name_C= sulinman
```



Thank you